

(11) [20] PostgreSQL 벡터 데이터 관리의 근본적인 제한 사항

2026-03-19 : Exqutor : “Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL” (ICDE 2024, Purdue University)

1. 개요

1.1 연구 배경

본 연구는 다음과 같은 질문을 다룬다:

“PostgreSQL에서 벡터 데이터를 효율적으로 관리하는 것은 가능한가?”

가 :

- Yes,** PostgreSQL은 벡터 데이터베이스로 사용될 수 있다.
- No,** PostgreSQL은 벡터 데이터를 지원하지 않는다.
- Yes and No,** PostgreSQL은 벡터 데이터를 지원하지만, 몇 가지 제한 사항이 있다.

: 3. “PostgreSQL은 벡터 데이터를 지원하지만, 몇 가지 제한 사항이 있다.”

1.2 PostgreSQL 5.15 버전의 벡터 데이터 관리

PostgreSQL 5.15 버전에서는 pgvector가 Faiss(벡터 데이터베이스)와 통합되어 벡터 데이터를 관리할 수 있다.

#

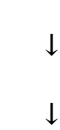
-
- 1
 - 2
 - 3
 - 4
 - 5

1.3 PostgreSQL의 벡터 데이터 관리

참고 1: PostgreSQL의 Buffer Pool은 메모리에서 데이터를 저장하는 데 사용된다.

PostgreSQL은 다음과 같은 제한 사항을 가진다:

PostgreSQL은 “(8KB) 블록 크기”를 사용한다.



ID →
 ↓
 (Latch) (spin lock,)
 ↓
 ↓

HNSW :

HNSW :

→ 가 가

→

→ (~)

(random access) . , .

: (Latch Contention)

HNSW 100 top-10 :

: ~

: - -

(spin lock):

: ~50ns

: ~500ns~1000ns

:

: 50ms (CPU)

: 100ms ()

: 66% 가

:

: 100 (1536 , OpenAI)

: top-10

PostgreSQL pgvector: 150ms

Faiss (): 8ms

: $150/8 = 18.75$

가 ?

- PostgreSQL

-

- , PostgreSQL

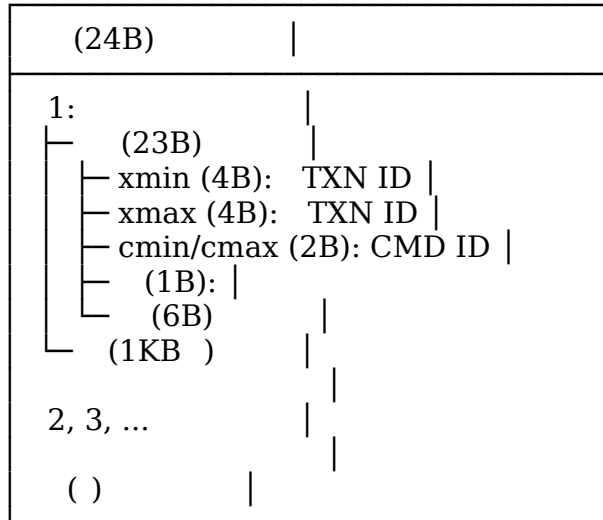
• DB 가

가 : .VBASE()

2: Tuple Access — MVCC

PostgreSQL :

8KB :



:

1 : ()

2 :

- xmin, xmax, cmin, cmax
- ?

3 : MVCC 가

- : TXN 1000
- TXN 800 , TXN 900
- TXN 1000 ? YES/NO
- 5~10 가

4 :

-
- (1KB)

:

1 :

: $1536 \text{ float} \times 4\text{B} = 6.144\text{KB}$

MVCC : (200B) + (10 × 8B = 80B)
가 : 280B (= 4% 가)

100 :
: 100ms
+ MVCC : 20~30ms
: 120~130ms

가 ?

- MVCC PostgreSQL (/)
- 가
- DB

가 : . MVCC 가

3: (Space Amplification) — 8KB 8KB

PostgreSQL :

HNSW :
: 1536 (6KB)
: 40~50 (200B)

PostgreSQL 가 :

1 200B
→ 7,800B
→ : 97.5%

가 :
: 100MB
PostgreSQL : 800MB (8)

:
: 1,000 (1536D)
: 60GB

Faiss HNSW :
: ~90GB
()
: ~100ns ()

PostgreSQL HNSW :
: ~480GB (5.3 !)

(SSD)
: ~10,000ns (SSD) = 100

:

PostgreSQL :

1.
 - 8KB
 - 가 가
2. MVCC
 - xmin/xmax/cmin/cmax = 24B
 - 1000 × 24B = 240MB 가
3.
 -
 - maintenance_work_mem
4.
 - 23B
 - 1000 × 23B = 230MB 가

가 ?

- 8KB PostgreSQL (, , WAL)
- I/O 가
- 가
- DB 가

가 : .

4: — WAL

PostgreSQL :

:

1 :
└ → -

2 : WAL(Write-Ahead Log)
└ " " (I/O)

3 : HNSW

├
├ 가 가 K
├
└

└ + WAL

4 : maintenance_work_mem

└ (- SSD) →

: 1 = 20~50 I/O

:

: 1000 (1536D, 60GB)

PostgreSQL pgvector:

: 48

: 57 /

Faiss:

: 30

: 5,500 /

: 5,500 / 57 ≈ 96

:

Faiss: (I/O)

pgvector: 1 = 20 I/O (WAL)

→ 1000 × 20 = 2 I/O

→ 48

WAL :

:

A: 0~500 ()

B: 500

└

:

- WAL : "500 "

-

- WAL 가

-

가 ?

• WAL ()

• WAL ()

• (가)

가 : .

예 5: 고정 선택도(Fixed Selectivity) — 쿼리 실행 계획

PostgreSQL :

WHERE :

WHERE age > 30

→

→ : ~90%

WHERE name = 'John'

→

→ : ~95%

:

WHERE embedding <-> query_vec < 0.5

pgvector :

→ " "

→ : = 33.3%

: 1/3 가

:

: Wikipedia (1000)

: < 1.0 (가 100)

: 0.001% (100 / 10,000,000)

pgvector : 33.3%

: 33,300 !

: < 0.1 (가 1)

: 0.00001%

pgvector : 33.3%

: 3,330,000 !

:

:

SELECT o.order_id, l.product_id, l.price

FROM orders o

JOIN lineitem l ON o.id = l.order_id

WHERE l.product_embedding <-> query_vec < 0.5

:

orders: 100

lineitem: 600

pgvector ():

1. product_embedding <-> query_vec < 0.5 = 33.3%
 → lineitem : 600 × 33.3% = 200

():
 < 0.5 = 0.001%
 → lineitem : 600 × 0.001% = 60

pgvector ():
 1. orders Seq Scan: 100
 2. lineitem Nested Loop Join
 → 600 × 100 = 6
 3. : 200

:
 1. lineitem : 60
 2. orders : 60 × 1
 3. : 60

:
 pgvector: 6 → 850
 : 60 → 2
 : 425

가 ?

:
 -
 -
 - 가 ()

:
 - (Exqutor)
 - ()
 - (ECQO)

가 : . Exqutor 가 가

1.4 vs :

vs :

():- : PostgreSQL - : MVCC - : 8KB

PostgreSQL PostgreSQL 가 .

:- : WAL () + - :

, 가 .

1.5 検索 実行 時間

- :
 - 1.
 - pgvector/PASE C
 -
 -
 - 2.
 - :
 - : 150ms
 - : 140ms (→ 10ms)
 - MVCC : 120ms (→ 20ms)
 - WAL : 100ms (→ 20ms)
 - : 8ms (→ 92ms)
 - 3.
 - PostgreSQL
 - ,
- : - SIFT-128M: 1.28 (128D) - GloVe: 119 (300D) - MS COCO: 123 (2048D)
-

2. 検索 実行 時間 (メモリ)

2.1 検索 実行 時間 (Buffer Pool) 検索?

- :
- :
- L1 : 0.5ns (64KB)
- L2 : 7ns (256KB)
- L3 : 40ns (8MB)
- : 100ns (16GB)
- SSD: 100,000ns (1TB) ← 1000
- HDD: 10,000,000ns (10TB) ← 100,000
- :
- PostgreSQL :
- : shared_buffers = 4GB
- PostgreSQL

- :
1. (8KB)
 - 2.
 3. ()
 - 4.

:

95% : $= 100\text{ns} \times 0.05 + 100,000\text{ns} \times 0.95 \approx 95,000\text{ns}$

: 100,000ns

→ 1.05 (!)

가 ?

: - (Random Access): - : ~ - : (50%)

SQL ():

0, 1, 2, 3, 4, ...

: 99% (1,2,3,4)

(HNSW):

500, 10000, 25, 50000, 1, ... ()

: 30~50% ()

2.2 락(Latch)? Spin Lock 락 락

:가 ?

A: 100 ...

B: 100

C: 100

→ !

: (Latch)

:

A:

1. ()
- 2.
3. ()

B:

1. ... (A 가)
2. A 가
- 3.
- 4.
- 5.

```

:
- ( )
- ( )
- ( )

```

Spin Lock :

: Spin Lock

```

:
while (가 busy) {

    // CPU 가 (spinning)
}
    = busy

```

```

:
- busy-wait: CPU
- 가 : /
- (sleep )

```

```

:
- sleep (futex)
- spin lock:

```

```

:
HNSW :
100,000

```

```

:
: 50ns
: 100ns
: 50ns
: 200ns/

```

```

:
: 50ns
: 500ns ( )
: 50ns
: 100ns
: 50ns
: 750ns/

```

```

: 750/200 = 3.75
100 : 100 × 0.55ns = 550ms

```

2.3 MVCC (Multi-Version Concurrency Control)

:

A (TXN 100):
SELECT age FROM users WHERE id = 1

T0: age = 30

B (TXN 101): [TXN 100]
UPDATE users SET age = 31 WHERE id = 1

T1: age = 31

A:
T2: SELECT age FROM users WHERE id = 1
age = ? (30 가 31 가?)

: MVCC

:

ID=1:

1: age=30, xmin=50, xmax=100
2: age=31, xmin=101, xmax= ∞

A (100):
TXN 100 100 TXN " "

1 : xmin(50) < 100 xmax(100) ≤ 100
→ TXN 100
→

2 : xmin(101) > 100
→ TXN 100
→

: TXN 100 age=30

B:
101: TXN < 101
2 : xmin(101) = 101, xmax(∞)
→
→ age=31

가 ?

1 :

1. : MVCC 가
 -
 - xmin/xmax
 - 5~10
 - 100ns 가
2. 100 :
 - $100 \times 100\text{ns} = 100\text{ms}$ 가
3. :
 - : 100ms
 - MVCC : 100ms
 - : 50%

DB :

:

-
-
-

:

- " "

- MVCC

-

- xmin/xmax

2.4 WAL (Write-Ahead Log)

:

:

:

A - - -

:

!

→

: WAL

:

1. A
2. WAL " A " ()

- 3.
- 4.
- 5.

:

WAL
→ : WAL →

:
" (Write-Ahead), "

:

:

- 1.
2. WAL " "
- └ fsync() :
- └ : 1~10ms ()
- 3.
4. (2)

:
I/O 가
 $1000 \times 5\text{ms} = 50,000 = 13.8$

:

:

- ():
1. (WAL)
 2. HNSW (WAL)
-

- ():
1. ()
 2. HNSW
 - 3.

→ ()

:

-

- ()

2.5 内存 索引 内存

8KB :

HNSW :

$$\begin{aligned} 1 &= 6KB (1536D \times 4B) \\ &= 200B (50 \times 4B) \end{aligned}$$

: 6.2KB

PostgreSQL (8KB):

: 24B	
: 23B	
: 6.2KB	
: 1.7KB (!)	

→ : 1.7KB / 8KB = 21%

:
(200B):

: 24B	
: 23B	
: 200B	
: 7.7KB (!)	

→ : 7.7KB / 8KB = 96%!

:

1000 HNSW :

Faiss:

: 60GB ()
HNSW : 30GB (50 ×)
: 90GB

PostgreSQL:

: 90GB / 0.8 = 112.5GB (20%)
MVCC : 240MB
: 100GB ()
: 200GB
- SSD

:

Faiss: 100ns
PostgreSQL: SSD 100,000ns (1000)

2.6

가 ?

(가):

: [10, 15, 20, 25, 30, 35, 40, 45, 50, ...]
: WHERE age > 30
:
→ 30
→ 90%

(가):

: 1536 (sphere)
: WHERE embedding <-> query_vec < 0.5

:
1 (query_vec_a): 100 (0.001%)
2 (query_vec_b): 1000 (0.01%)
3 (query_vec_c): 100 (0.001%)

→

:

가 ()
()

2.7 Faiss vs PostgreSQL

Faiss:

:
-
- ()
- MVCC ()
- WAL ()
- 8KB (가)
- ()

:

- ()
- ACID
- SQL
-

PostgreSQL:

```

:
- ( - )
- MVCC ( / )
- WAL ( )
- 8KB ( )
- ( )

```

$$\vdots$$

- SQL
- ACID

•
•
—

→ pgvector = vs

2.8 “가”와 “가나”의 차이

$$\vdots$$

1.
 -
 - : (PostgreSQL)

2.

- $\rightarrow \text{MVCC} \rightarrow$
- $\text{MVCC} \rightarrow$

3. —
—
:

1. $\quad - \quad : \quad \text{WAL}$

- WAL ,
2. 가
- :
 - → /
3. 가
- : (Exqutor)
 -
-

3. Exqutor 실행 과정

3.1 5 가 실행 과정 Exqutor 실행

5: (Fixed Selectivity) — Exqutor 가 ** **

```
:
pgvector = 33.3% 가
→
→
→ (425 )
```

Exqutor :
ECQO (Estimated Cardinality Query Optimization):

- 1.
- 2.
- 3.

```
:
- pgvector 33.3% → 가 (: 0.1%)
-
- 425 가
```

1~4: — Exqutor 가 ** **

1 ():

-
- 20
- /

2 (MVCC):

- xmin/xmax
- 30%

3 ():

- 8KB
- 8

4 ():

- WAL ()
- 96

3.2 □□□ □□ □□ □□

Exqutor (pgvector):

```

:
( 5):
→ 425 (가 )

( 1):
→ 20

MVCC ( 2):
→ 1.3

I/O ( 3):
→ 1

( 4):
→ ( )

( ):
425 × 20 × 1.3 = 11,050

:
Faiss: 100ms
pgvector: 1,100 (11,000 )

```

Exqutor :

```

5 : 425 → 1 ( )

:
( 1): 20
MVCC ( 2): 1.3
( 3):
( 4): N/A

:
1 × 20 × 1.3 = 26

:
pgvector + Exqutor: 2,600ms
: 100ms × 26 = 2,600ms

:

```

Faiss: 100ms
pgvector: 1,100
pgvector + Exqutor: 2~5

:
1,100 / 2.5 = 440 !
: 1,000 - 20 = 50

3.3 実行時間: 実行時間

:

```
SELECT
  o.order_id,
  o.customer_id,
  COUNT(*) as item_count
FROM orders o
JOIN lineitem l ON o.id = l.order_id
WHERE l.product_embedding <-> query_vec < 0.5
GROUP BY o.id, o.customer_id
```

: - orders: 100 - lineitem: 600

pgvector ():

(): 33.3%
WHERE = 33.3%
→ lineitem : 600 × 0.333 = 200

:

1. orders Seq Scan (100)
2. Nested Loop Join
- orders lineitem (600)
- 600 × 100 = 6
3. WHERE : 6 × 0.333 = 2
4. GROUP BY: 2

:

- 6 :
- : 850

pgvector + Exqutor:

:

1 : (pgvector)
- 200

2 :
- WHERE : 100 (0.0016%)
- : 200

- : 20,000 !

3 :

- lineitem (100)
- orders (100)

:

1. lineitem (WHERE) → 100
2. orders → 100

:

- 100
- : 2

: $850 / 2 = 425$!

3.4 **Exqutor** **Exqutor** **Exqutor** **Exqutor** **Exqutor**

:

1. **pgvector** ()
 - Faiss 100~1000
2. (5 가)
 - 5: (425)
 - 1: (20)
 - :
3. ()
 - 5: /가
 - 4: WAL

Exqutor :

: " "
→ 425 가

Exqutor: " "
→
→ 425 ()

:

:

- pgvector
- Exqutor

:

-
-
- Exqutor

4. Q&A

Q1: 어떤 것들이 있을까?

A: 가 :

1. **pgvector** 가 ?
 - Faiss 100~1,000 ()
 2. 가 ?
 - 5 가 :
 - (20)
 - MVCC (30%)
 - (8)
 - (96)
 - (425)
 3. 까 ?
 - . (, MVCC)
 - (,)
-

Q2: “가”와 “가”의 차이?

A: :

(가):- - : → MVCC →

(가):- - 까 - : WAL - : WAL () - : () -

Q3: 어떤 것들이 있을까?

A: :

?

:

: 100ns

SSD: 100,000ns (1000)

:

가 ?

SQL:

SELECT * FROM users WHERE age > 30

:
0 → 1 → 2 → 3 → ...
: 99% ()

:

:

HNSW

:
500 → 10000 → 25 → 50000 → 1 → ...
: 30% ()

:

→ -
→ 100ns vs 100,000ns = 1000

Q4: 락(Latch)가 뭐예요?

A: :
:

:

A: (1)
B:
→ (A)

: - () - Spin lock (CPU 가) - ()

:

HNSW : 100,000

: 200ns/
: 750ns/ (3.75)

:
가 : 550ms
150ms

Q5: MVCC 가 무엇? 어떤 이 무엇?

A: MVCC = /

:

(MVCC):
TXN A ():
TXN B () ()
TXN A: ... (TXN B)

MVCC ():
TXN A () : " " ()
TXN B () : " " ()
→ /

:

:

xmin: ID (4B)
xmax: ID (4B)
cmin/cmax: ID (2B)
10B

:

1.
2.
3. " ?"
→ 가 CPU/

:

:

-
-
-

:

- MVCC
-
-

Q6: 어떤 어떤 8KB 무엇?

A: , :

1: 8KB

HNSW :
: 6KB
: 200B
: 6.2KB

8KB :
├ : 24B
├ : 23B
├ : 6.2KB
└ : 1.7KB ()

: 21%

:
└ : 7.7KB (!)
: 96%

2: MVCC

1000 × 24B = 240MB 가

3:

:

100GB ()

:

Faiss (): 90GB

PostgreSQL: 200GB

: $200/90 \approx 2.2$

:

Faiss:

PostgreSQL: → SSD

:

- Faiss: 100ns

- PostgreSQL: 100,000ns (SSD)

- : 1000 !

Q7: ☐ ☐ ☐ 96 ☐ ☐ ☐ ☐ ?

A: .WAL :

:

1 :

1 :

2 : WAL (I/O)

→ 1~10ms ()

3 :

→ 1 = 5~10ms

1000 :

10ms × 1000 = 100,000,000ms = 11,111

:

48 = 96 (20ms)

:

Faiss (): 30 (0.0018ms/)

():

1: WAL

-

- WAL ,

- 50

2:

-

- 4 (4)

3:

-

- WAL

- I/O

- 10

Q8: □□□□ 33.3%□□□ □□ □ □□□□□?

A: :

:

:

SELECT * FROM lineitem

WHERE product_embedding <-> query_vec < 0.5

: 100

pgvector : 600 × 33.3% = 200

: 20,000 !

```

:
SELECT o.order_id
FROM orders o (100 )
JOIN lineitem l (600 )
  ON o.id = l.order_id
WHERE l.embedding <-> query_vec < 0.5

```

```

pgvector      : 33.3%
→ lineitem    200

```

```

1 (pgvector ):
orders Seq Scan (100 )
└─ Nested Loop Join
   └─ lineitem (600 )

```

```

: 600 × 100 = 6
WHERE : 6 × 0.333 = 2
: 850

```

```

2 ( ):
lineitem (WHERE) → 100
orders → 100

```

```

: 2

```

```

: 850 / 2 = 425 !

```

가 ?

```

:
1: < 0.5 → 100
2: < 0.5 → 10,000

```

가
(Exqutor)

Q9: pgvector 가 SQL + 가

A: :

pgvector :

1.
 - SQL + 가
 - 5%

2.
 - < 100
 - Faiss ()
 - 가

3. SQL
 - +
 - /
 - PostgreSQL

4. ACID
 - /
 - Faiss () 가

pgvector :

1.
 - API
 -
 - 1

2. ()
 - → SSD
 -

3.
 -
 -

4.
 - =
 - 100ms vs 1000ms 가

:

pgvector :

- +
- SQL

.

:

- Faiss/Milvus (DB)
- pgvector + Exqutor ()

Q10: Faiss 가 1000000 Faiss 100 100 1000?

A: Faiss :

Faiss :

1.

-
- 1000 = ~90GB
- 512GB ,

2.

- /가
-
-

3. SQL

- "embedding < 0.5 AND created_at > '2024-01-01'"
-
-

4. ACID

-
-
- 가

5.

-
-

pgvector :

1.

-
- SSD/HDD 가

2.

- /가
-

3. SQL

-
- WHERE/JOIN/GROUP BY

4. ACID

-
- /

5. (Citus)

- PostgreSQL

:

Faiss vs pgvector:

- Faiss " ", "
- pgvector " ", "

:

- : Faiss
 - + SQL : pgvector
 - + SQL : pgvector + Exqutor
-

Q11: Exqutor 5 가 4 4 4 4 4?

A: 5 () :

5 :

: ECQO (Estimated Cardinality Query Optimization)

:

- 1.
- 2.
- 3.
4. →

:

- : 33,300 → 0 ()
- : 425

:

- 1 ():
 - Exqutor 가
 - 20

2 (MVCC):

-
- 30%

3 ():

-
- 8

4 ():

- (Exqutor)
 - 96
-

Q12: 어떤 어떤 어떤 어떤?

A: :

1 ():

: 가

:

- VBASE:

()

-

- PostgreSQL

:

- 20 - 1

2 (MVCC):

: 가

:

- immutable

- MVCC (가)

-

:

- 30% - 5%

3 ():

: 8KB

:

-

- 가 ()

- (PostgreSQL)

:

- 8 - 2

4 ():

:

1: WAL ()

-

- 50

2:

-

- 4 (4)

3:

-
- 10

: 96 → 5 개

Exqutor :

Exqutor " 5 "
- 425 (가)
-

:
- ()
- 가
-

:
pgvector (): 1,000
pgvector + Exqutor: 20
pgvector + Exqutor + : 5 가

5. (5 개)

. Exqutor [] .

“PostgreSQL ” (ICDE 2024, Purdue University) .

pgvector ?

. PostgreSQL pgvector Faiss **100 1000** .

:
- Faiss (): 100ms
- pgvector (): 1,100
- : 11,000

“ ” , ? .

5 가

5 가 .

: **(Latch)**

PostgreSQL “ ” . (HNSW) . / .

: **20**

:

PostgreSQL xmin, xmax . MVCC(/) , .
: **30%**

:

PostgreSQL 8KB . HNSW . 1000 가 Faiss 90GB PostgreSQL 200GB .

: **8** , **SSD** **1000**

:

WAL(Write-Ahead Log) . 1000 48 (Faiss 30) .

: **96**

:

가 . PostgreSQL () “33.3%”가 . 0.001% ~ 1% .

: 1000 가 100
- pgvector : 333 (33.3%)
- : 100 (0.001%)
- : 33,300 !

. 100 orders 600 lineitem , pgvector 6 (100).

: **425**

□□: □□ vs □□

가 :

():- ()- MVCC ()- (8KB)

PostgreSQL .

:- (WAL)- (/가)

, ㄱㅈ .

Exqutor □ □□

Exqutor 가 . ECQO(Estimated Cardinality Query Optimization) .

: 1. 2. 3.

:- : 33,300 → 0 - : 425 - : pgvector 1,100 → 2~5

(, MVCC) 20 , 가가ㅈ () **50** .

□□

: 1. **pgvector** 2. , 가 3. 가

Exqutor : 1. 가 (5) 2. 3. 가
 , pgvector 가 “ ” . “ ” , Exqutor 가 .
 .

□□

A. □□□□ □□

- **MVCC:** /
- **WAL:** Write-Ahead Log,
- **Latch:** ,
- **Selectivity:** %
- **Cardinality:**
- **HNSW:** Hierarchical Navigable Small World,
- **ECQO:** Exqutor ,

B. □ □□ □□□

- :
1. Faiss ?
 - Faiss , PostgreSQL
 - (가)
 2. ?
 -
 -
 3. pgvector ?
 - 0.5.x
 - 0.7.x

Exqutor :

1.
 - , MVCC,
 -
2.
 -
 -
3. 가
 - MySQL, Oracle ?
 -